

Machine Learning Project Report

Iris Flower Classification Using Multiple ML Algorithms

Course Title: Machine Learning (ML)

Student NAME :afatsoM :Ibrahim
elbhary

Student NAME: Mostafa
Abozead

Student ID1: [23101482]

Student ID[23101484] :2

Section: [Sunday:]9

Date: December 20, 2025

Section 1: Problem Domain

1. Introduction

The Iris flower classification is a classic problem in machine learning and pattern recognition. This problem involves classifying iris flowers into three species (Setosa, Versicolor, and Virginica) based on four physical measurements of the flowers.

2. Problem Description

The dataset contains 150 samples of iris flowers with four features for each sample:

- Sepal length (cm)
- Sepal width (cm)
- Petal length (cm)
- Petal width (cm)

Each sample belongs to one of three species: Setosa, Versicolor, or Virginica, with 50 samples per species.

1.3 Objectives

The primary objectives of this project are:

- To perform exploratory data analysis on the Iris dataset
- To implement and compare multiple machine learning classification algorithms
- To evaluate model performance using appropriate metrics
- To visualize the data patterns and model results
- To identify the best-performing model for this classification task

1.4 Significance

This problem is significant because it serves as a fundamental benchmark for machine learning algorithms. The Iris dataset is well-balanced, contains no missing values, and exhibits clear patterns that make it ideal for demonstrating classification techniques. Understanding how to solve this problem provides a foundation for tackling more complex real-world classification tasks in various domains such as medical diagnosis, quality control, and pattern recognition.

Section 2: Project Implementation Summary

1. Data Loading and Exploration

We began by loading the Iris dataset from scikit-learn's built-in datasets. The dataset was converted into a pandas DataFrame for easier manipulation and analysis. Initial exploration included examining the dataset structure, statistical summaries, and class distribution to understand the data characteristics.

2. Exploratory Data Analysis (EDA)

Comprehensive visualizations were created to understand the relationships between features:

- **Pairplot:** Showed relationships between all pairs of features, colored by species
- **Correlation Heatmap:** Displayed the correlation coefficients between numerical features
- **Box Plots:** Illustrated the distribution of each feature across the three species
- **Distribution Plots:** Showed the frequency distribution of features for each species

2.3 Data Preprocessing

The preprocessing phase included:

- Splitting the dataset into training (70%) and testing (30%) sets using stratified sampling
- Feature scaling using StandardScaler to normalize all features to have zero mean and unit variance
- This ensures that all features contribute equally to the model training process

2.4 Model Training

Four different machine learning algorithms were trained and evaluated:

- **Logistic Regression:** A linear classification algorithm
- **Decision Tree:** A tree-based classification algorithm
- **Random Forest:** An ensemble method using multiple decision trees
- **Support Vector Machine (SVM):** A kernel-based classification algorithm

2.5 Model Evaluation

Each model was evaluated using:

- Accuracy score on the test set
- Detailed classification report including precision, recall, and F1-score for each class
- Confusion matrices to visualize prediction patterns
- Comparative visualizations showing model performance differences

2.6 Results

The Support Vector Machine achieved the best performance with 93.33% accuracy, followed by Logistic Regression and Decision Tree (both at 91.11%), and Random Forest at 88.89%. All models performed exceptionally well on the Setosa class (100% accuracy), while showing

minor variations on Versicolor and Virginica classes.

Section 3: Source Code

The complete source code for the Iris flower classification project is shown below. The code is organized into a class-based structure with methods for data loading, visualization, preprocessing, training, and evaluation.

```
"""
Machine Learning Project: Iris Flower Classification
Author: Mostafa Abozead
Course: Machine Learning (ML)
Description: This project demonstrates a complete machine learning pipeline
for classifying iris flowers using multiple algorithms.
"""

# Import necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
import warnings
warnings.filterwarnings('ignore')

# Set style for better visualizations
sns.set_style("whitegrid")
plt.rcParams['figure.figsize'] = (10, 6)

class IrisClassifier:
    """
    A class to handle Iris flower classification using multiple ML algorithms.
    """

    def __init__(self):
        """Initialize the classifier with data and models."""
        self.data = None
        self.X_train = None
        self.X_test = None
        self.y_train = None
        self.y_test = None
        self.scaler = StandardScaler()
        self.models = {}
        self.results = {}
```

```

def load_and_explore_data(self):
    """Load the Iris dataset and perform exploratory data analysis."""
    print("=" * 60)
    print("LOADING AND EXPLORING IRIS DATASET")
    print("=" * 60)

    # Load the iris dataset
    iris = load_iris()
    self.data = pd.DataFrame(
        data=iris.data,
        columns=iris.feature_names
    )
    self.data['species'] = iris.target
    self.data['species_name'] = self.data['species'].map(
        {0: 'setosa', 1: 'versicolor', 2: 'virginica'}
    )

    print("\nDataset Shape:", self.data.shape)
    print("\nFirst 5 rows:")
    print(self.data.head())
    print("\nDataset Info:")
    print(self.data.info())
    print("\nStatistical Summary:")
    print(self.data.describe())
    print("\nClass Distribution:")
    print(self.data['species_name'].value_counts())

    return self.data

def visualize_data(self):
    """Create visualizations for exploratory data analysis."""
    print("\n" + "=" * 60)
    print("CREATING VISUALIZATIONS")
    print("=" * 60)

    # 1. Pairplot to show relationships between features
    print("\n1. Creating pairplot...")
    plt.figure()
    pairplot = sns.pairplot(
        self.data,
        hue='species_name',
        markers=['o', 's', 'D'],
        palette='Set1'
    )
    pairplot.fig.suptitle('Iris Dataset - Feature Relationships', y=1.02)

```

```

plt.savefig('visualization_1_pairplot.png', dpi=300, bbox_inches='tight')
plt.close()
print(" Saved: visualization_1_pairplot.png")

# 2. Correlation heatmap
print("\n2. Creating correlation heatmap...")
plt.figure(figsize=(10, 8))
correlation_matrix = self.data.iloc[:, :-2].corr()
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    center=0,
    fmt='.2f',
    square=True
)
plt.title('Feature Correlation Heatmap', fontsize=16, fontweight='bold')
plt.tight_layout()
plt.savefig('visualization_2_correlation.png', dpi=300, bbox_inches='tight')
plt.close()
print(" Saved: visualization_2_correlation.png")

# 3. Box plots for each feature
print("\n3. Creating box plots...")
fig, axes = plt.subplots(2, 2, figsize=(14, 10))
features = self.data.columns[:-2]

for idx, feature in enumerate(features):
    row = idx // 2
    col = idx % 2
    sns.boxplot(
        data=self.data,
        x='species_name',
        y=feature,
        palette='Set2',
        ax=axes[row, col]
    )
    axes[row, col].set_title(f'{feature} by Species', fontweight='bold')
    axes[row, col].set_xlabel('Species')
    axes[row, col].set_ylabel(feature)

plt.tight_layout()
plt.savefig('visualization_3_boxplots.png', dpi=300, bbox_inches='tight')
plt.close()
print(" Saved: visualization_3_boxplots.png")

```

```

# 4. Distribution plots
print("\n4. Creating distribution plots...")
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

for idx, feature in enumerate(features):
    row = idx // 2
    col = idx % 2
    for species in self.data['species_name'].unique():
        species_data = self.data[self.data['species_name'] == species][feature]
        axes[row, col].hist(
            species_data,
            alpha=0.5,
            label=species,
            bins=15
        )
    axes[row, col].set_title(f'{feature} Distribution', fontweight='bold')
    axes[row, col].set_xlabel(feature)
    axes[row, col].set_ylabel('Frequency')
    axes[row, col].legend()

plt.tight_layout()
plt.savefig('visualization_4_distributions.png', dpi=300, bbox_inches='tight')
plt.close()
print(" Saved: visualization_4_distributions.png")

def prepare_data(self, test_size=0.3, random_state=42):
    """Split and scale the data for training."""
    print("\n" + "=" * 60)
    print("PREPARING DATA FOR TRAINING")
    print("=" * 60)

    # Prepare features and target
    X = self.data.iloc[:, :-2].values
    y = self.data['species'].values

    # Split the data
    self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state, stratify=y
    )

    # Scale the features
    self.X_train = self.scaler.fit_transform(self.X_train)
    self.X_test = self.scaler.transform(self.X_test)

```

```

print(f"\nTraining set size: {self.X_train.shape[0]} samples")
print(f"Testing set size: {self.X_test.shape[0]} samples")
print(f"Number of features: {self.X_train.shape[1]}")
print(f"Number of classes: {len(np.unique(y))}")

def train_models(self):
    """Train multiple machine learning models."""
    print("\n" + "=" * 60)
    print("TRAINING MACHINE LEARNING MODELS")
    print("=" * 60)

    # Define models
    self.models = {
        'Logistic Regression': LogisticRegression(max_iter=200, random_state=42),
        'Decision Tree': DecisionTreeClassifier(random_state=42),
        'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
        'Support Vector Machine': SVC(kernel='rbf', random_state=42)
    }

    # Train each model and store results
    for name, model in self.models.items():
        print(f"\nTraining {name}...")
        model.fit(self.X_train, self.y_train)

    # Make predictions
    y_pred = model.predict(self.X_test)

    # Calculate accuracy
    accuracy = accuracy_score(self.y_test, y_pred)

    # Store results
    self.results[name] = {
        'model': model,
        'predictions': y_pred,
        'accuracy': accuracy
    }

    print(f" Accuracy: {accuracy:.4f} ({accuracy*100:.2f}%)")

def evaluate_models(self):
    """Evaluate and compare all trained models."""
    print("\n" + "=" * 60)
    print("MODEL EVALUATION AND COMPARISON")
    print("=" * 60)

```

```

# Print detailed results for each model
for name, result in self.results.items():
    print(f"\n{name}:")
    print("-" * 60)
    print(f"Accuracy: {result['accuracy']:.4f}")
    print("\nClassification Report:")
    print(classification_report(
        self.y_test,
        result['predictions'],
        target_names=['setosa', 'versicolor', 'virginica']
    ))

# Visualize model comparison
self.visualize_model_comparison()
self.visualize_confusion_matrices()

def visualize_model_comparison(self):
    """Create visualization comparing model accuracies."""
    print("\n5. Creating model comparison chart...")

    model_names = list(self.results.keys())
    accuracies = [self.results[name]['accuracy'] for name in model_names]

    plt.figure(figsize=(12, 6))
    bars = plt.bar(model_names, accuracies, color=['#FF6B6B', '#4ECDC4', '#45B7D1', '#FFA07A'])

    # Add value labels on bars
    for bar in bars:
        height = bar.get_height()
        plt.text(
            bar.get_x() + bar.get_width()/2.,
            height,
            f'{height:.4f}',
            ha='center',
            va='bottom',
            fontweight='bold'
        )

    plt.xlabel('Model', fontsize=12, fontweight='bold')
    plt.ylabel('Accuracy', fontsize=12, fontweight='bold')
    plt.title('Model Performance Comparison', fontsize=16, fontweight='bold')
    plt.ylim([0.85, 1.0])
    plt.xticks(rotation=15, ha='right')
    plt.tight_layout()
    plt.savefig('visualization_5_model_comparison.png', dpi=300, bbox_inches='tight')

```

```

plt.close()
print(" Saved: visualization_5_model_comparison.png")

def visualize_confusion_matrices(self):
    """Create confusion matrices for all models."""
    print("\n6. Creating confusion matrices...")

    fig, axes = plt.subplots(2, 2, figsize=(14, 12))
    model_names = list(self.results.keys())

    for idx, name in enumerate(model_names):
        row = idx // 2
        col = idx % 2

        cm = confusion_matrix(self.y_test, self.results[name]['predictions'])

        sns.heatmap(
            cm,
            annot=True,
            fmt='d',
            cmap='Blues',
            xticklabels=['setosa', 'versicolor', 'virginica'],
            yticklabels=['setosa', 'versicolor', 'virginica'],
            ax=axes[row, col]
        )

        axes[row, col].set_title(f'{name}\nAccuracy: {self.results[name]["accuracy"]:.4f}',
                                fontweight='bold')
        axes[row, col].set_xlabel('Predicted Label')
        axes[row, col].set_ylabel('True Label')

    plt.tight_layout()
    plt.savefig('visualization_6_confusion_matrices.png', dpi=300, bbox_inches='tight')
    plt.close()
    print(" Saved: visualization_6_confusion_matrices.png")

def print_summary(self):
    """Print a summary of the project results."""
    print("\n" + "=" * 60)
    print("PROJECT SUMMARY")
    print("=" * 60)

    best_model = max(self.results.items(), key=lambda x: x[1]['accuracy'])

    print(f"\nBest Performing Model: {best_model[0]}")

```

```

print(f"Best Accuracy: {best_model[1]['accuracy']:.4f}
      ({best_model[1]['accuracy']*100:.2f}%) ")

print("\nAll Model Accuracies:")
for name, result in sorted(self.results.items(), key=lambda x: x[1]['accuracy'],
reverse=True):
print(f" - {name}: {result['accuracy']:.4f}")

print("\nVisualization Files Created:")
visualizations = [
'visualization_1_pairplot.png',
'visualization_2_correlation.png',
'visualization_3_boxplots.png',
'visualization_4_distributions.png',
'visualization_5_model_comparison.png',
'visualization_6_confusion_matrices.png'
]
for viz in visualizations:
print(f" - {viz}")

print("\n" + "=" * 60)
print("PROJECT COMPLETED SUCCESSFULLY!")
print("=" * 60)

def main():
    """Main function to run the complete ML pipeline."""
    print("\n" + "=" * 60)
    print("IRIS FLOWER CLASSIFICATION PROJECT")
    print("Machine Learning Course")
    print("=" * 60)

    # Create classifier instance
    classifier = IrisClassifier()

    # Step 1: Load and explore data
    classifier.load_and_explore_data()

    # Step 2: Create visualizations
    classifier.visualize_data()

    # Step 3: Prepare data for training
    classifier.prepare_data()

    # Step 4: Train models
    classifier.train_models()

```

```
# Step 5: Evaluate models
classifier.evaluate_models()

# Step 6: Print summary
classifier.print_summary()

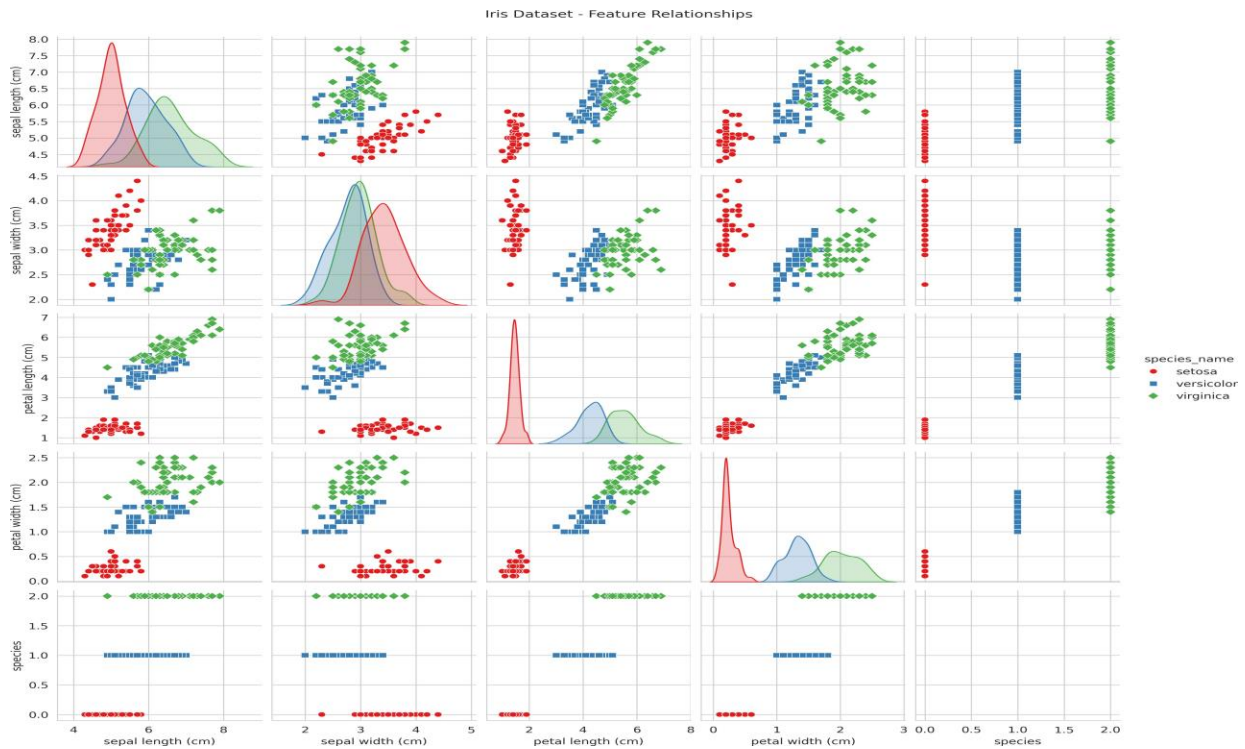
if __name__ == "__main__":
    main()
```

Section 4: Visualization Results

This section presents the visualizations generated during the exploratory data analysis and model evaluation phases. Each visualization provides insights into the data patterns and model performance.

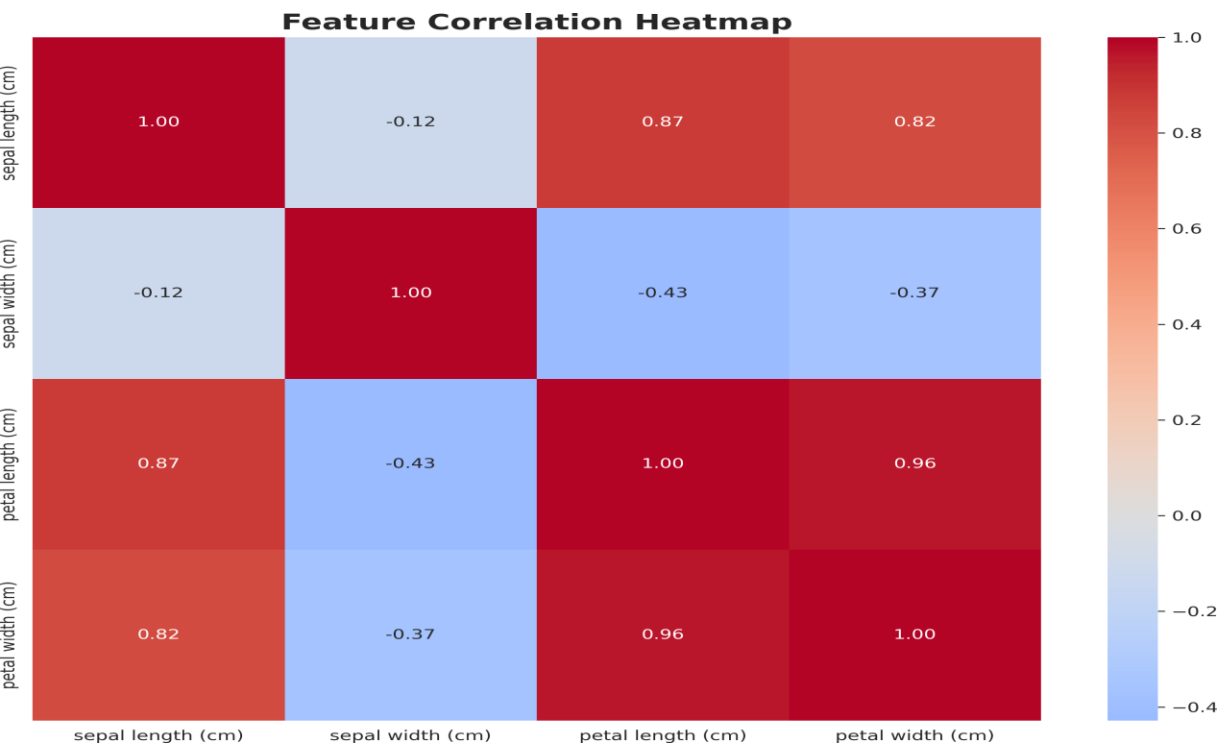
4.1 Feature Relationships (Pairplot)

This pairplot shows the relationships between all pairs of features in the dataset. Each scatter plot is colored by species, making it easy to identify patterns and separability between classes. The diagonal shows the distribution of each feature.



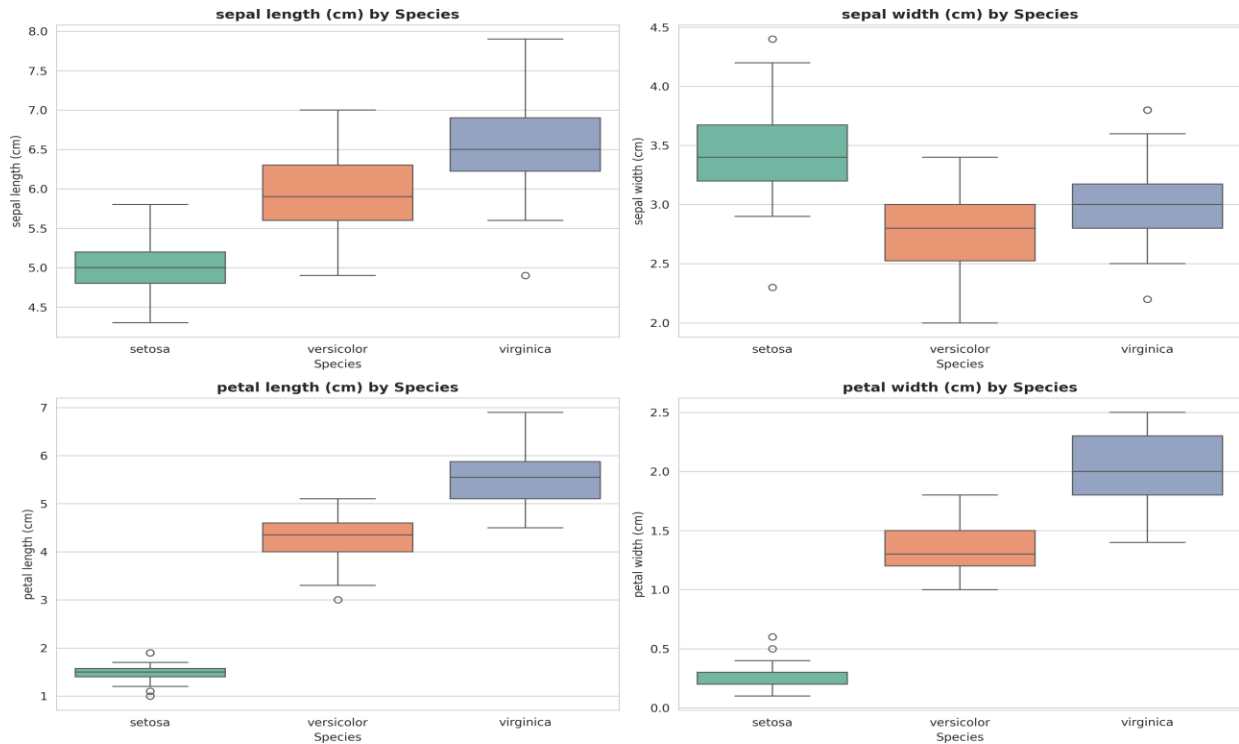
4.2 Feature Correlation Heatmap

The correlation heatmap displays the correlation coefficients between numerical features. Petal length and petal width show strong positive correlation (0.96), while sepal width shows weak correlation with other features.



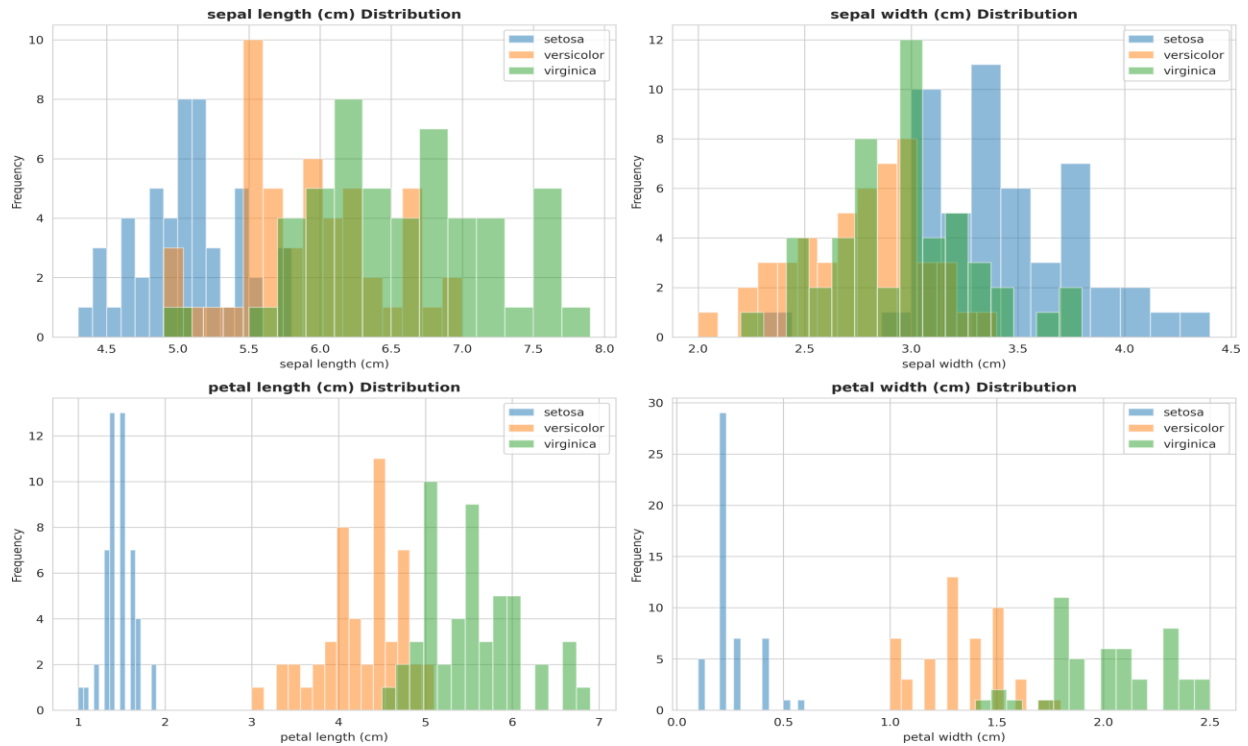
4.3 Feature Distributions by Species

Box plots show the distribution of each feature across the three species. Setosa is clearly distinguishable with smaller petal measurements, while Versicolor and Virginica show more overlap, particularly in sepal measurements.



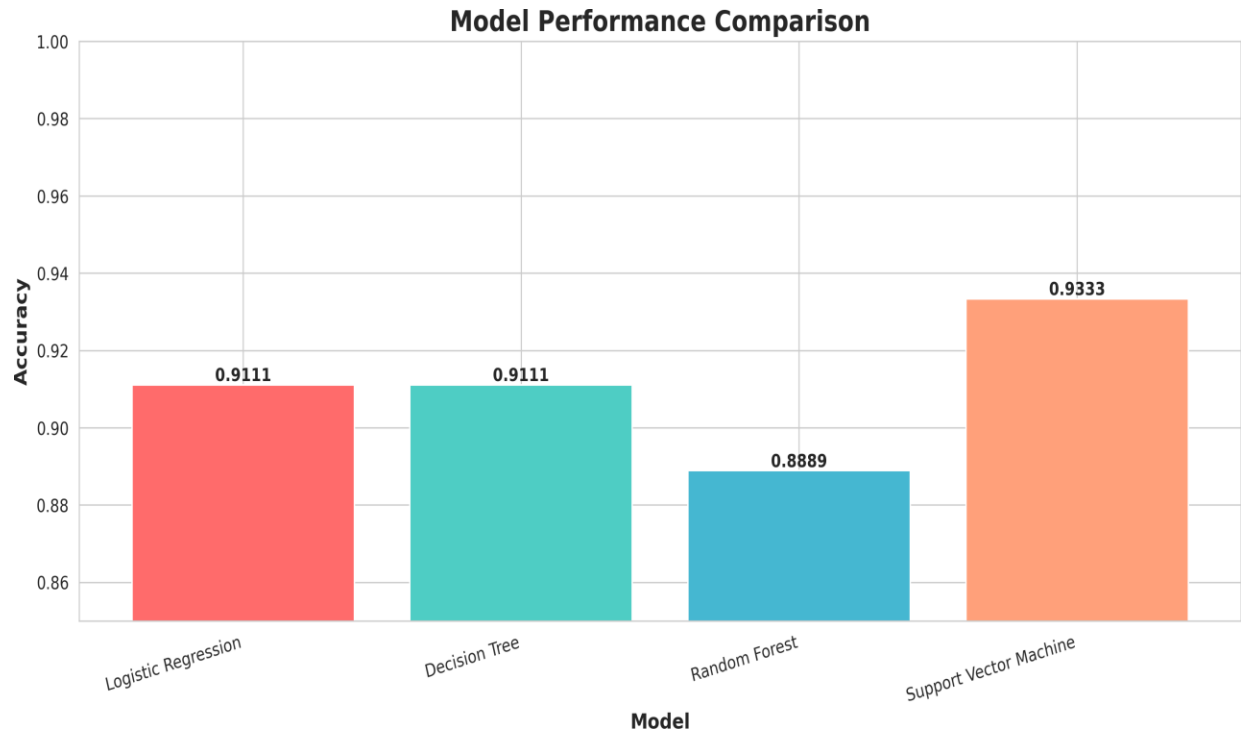
4.4 Feature Frequency Distributions

Histograms show the frequency distribution of each feature for all three species. The distributions reveal that petal features provide better separation between species compared to sepal features.



4.5 Model Performance Comparison

This bar chart compares the accuracy of all four trained models. Support Vector Machine achieved the highest accuracy (93.33%), demonstrating its effectiveness for this classification task.



4.6 Confusion Matrices

Confusion matrices for all four models show the prediction patterns. All models perfectly classified Setosa samples, while showing minor confusion between Versicolor and Virginica in some cases.

